Learning Objectives

Learners will be able to...

- Calculate Term Frequency-Inverse Document Frequency
- Create a TF-IDF-based model

info

Make Sure to Know

Intermediate Python.

Limitations

In this assignment, we only work with the NLTK library.

TF-IDF

Sometimes referred to as “weighted” bag of words, **Term Frequency-Inverse Document Frequency** solves the problem with scoring word frequency in a normal Bag of Words (BoW) model that highly frequent words start to dominate the document, but may not contain as much information as rarer but perhaps domain specific words.

TF-IDF rescales the frequency of words by how often they appear in all documents, so that the scores for frequent words like “the” that are also frequent across all documents are penalized.

Term Frequency: is a scoring of the frequency of the word in the current document.

$$TF(word, document) = \frac{count\ of\ word\ in\ the\ document}{number\ of\ words\ in\ the\ document}$$

Inverse Document Frequency: is a scoring of how rare the word is across documents.
The scores are a weighting where not all words are equally as important or interesting.

$$IDF(word) = log(\frac{number\ of\ documents}{number\ of\ documents\ that\ contain\ the\ word})$$

$$TF - IDF(word, document) = TF(word, document) * IDF(word)$$

TF-IDF scores have the effect of highlighting words that are distinct (contain useful information) in a given document.

Example calculation

Let’s try calculating the TF-IDF scores for a set of documents:

Document 1:
> The dog plays with the ball.

Document 2:
> The cat plays with the ball.

First, we calculate the Term Frequency for each word within their respective document:
$TF(the, document1) = \frac{2}{6} \approx 0.333$
$TF(dog, document1) = \frac{1}{6} \approx 0.167$

$TF(plays, document1) = \frac{1}{6} \approx 0.167$
$TF(with, document1) = \frac{1}{6} \approx 0.167$
$TF(ball, document1) = \frac{1}{6} \approx 0.167$

$TF(the, document2) = \frac{2}{6} \approx 0.333$
$TF(cat, document2) = \frac{1}{6} \approx 0.167$
$TF(plays, document2) = \frac{1}{6} \approx 0.167$
$TF(with, document2) = \frac{1}{6} \approx 0.167$
$TF(ball, document2) = \frac{1}{6} \approx 0.167$

Note that the Term Frequency for any words in documents where they do not exist is 0.

Second, we calculate the IDF for each word across the documents:
$IDF(the) = log(\frac{2}{2}) = 0$
$IDF(dog) = log(\frac{2}{1}) \approx 0.301$
$IDF(cat) = log(\frac{2}{1}) \approx 0.301$
$IDF(plays) = log(\frac{2}{2}) = 0$
$IDF(with) = log(\frac{2}{2}) = 0$
$IDF(ball) = log(\frac{2}{2}) = 0$

Finally, for a given word in a document, we can calculate the TD-IDF score by multiplying the TF with the IDF:

$TF - IDF(dog, document1) = 0.167 * 0.301 \approx 0.050$
$TF - IDF(cat, document2) = 0.167 * 0.301 \approx 0.050$

All other TF-IDF scores are zero because either the Term Frequency is zero for the word not existing in that document, or the IDF is zero for the word existing in every document.

You can see how with similar sentences, a raw count across documents can emphasize words without a lot of meaning (in this case, `the` is 4 of the 12 words) while TF-IDF scoring highlights words that are unique to documents.

Implementing TF-IDF

Now that we understand the math behind TF-IDF, let's build a model using TF-IDF scoring.

Notice the code in the left panel which is similar to the start of a normal bag of words implementation. The only change we are going to make is when we build the scoring piece.

For the calculation of IDF, we will need the total number of documents:

```
total_documents = len(documents)
```

We will also need a place to store our TF-IDF scores:

```
vectors = []
```

The rest of the calculations will take place in a nested loop where we will create a document-specific vector to track all our IDF values before adding it to our overall `vectors` variable:

```
for document in documents:
  doc_vec = np.zeros((len(vocabulary),)) # create vector of all
        0's
  index = 0 # tracks which vocabulary word we are on as vector
        index
  for word in vocabulary:
```

To calculate **Term Frequency** we need to find the number of occurences of the given word in the document, the total number of words in the document, and then divide the two values:

```
# calculate TF
occurences = len([token for token in document.split() if
        token.lower() == word])
words_in_doc = len(document.split())
tf = float(occurences/words_in_doc)
```

To calculate the **Inverse Document Frequency** we have already found the total number of documents, so we need to find the number of documents in which the word occurs, and take the log of their quotient:

```
# calculate IDF
occurences = 0
for inner_document in documents:
    if word in inner_document.lower():
      occurences += 1

idf = np.log10(total_documents/occurences)
```

Note that this nested loop puts us at some pretty terrible running times – there are ways to use more storage in exchange for making this calculation faster.

Finally, we calculate, store and print the **TF-IDF** values:

```
    # store IDF into the vector
    doc_vec[index] = tf * idf
    index += 1

  # inside outer loop but not inner loop
  vectors.append(doc_vec)

# outside both loops
print("{0} \n{1}\n".format(vocabulary,np.array(vectors)))
```

You should see output that looks similar to:

```
['ball', 'cat', 'dog', 'plays']
[[0.         0.         0.05017167 0.        ]
 [0.         0.05017167 0.         0.        ]]
```

▼ **Got lost? Something not working?**

Here is the entirety of the code:

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import numpy as np
import string

documents = [‘The dog plays with the ball.’, ‘The cat plays with the ball.’]

# Create Tokens from the set of documents
all_words = []
for document in documents:
all_words.extend(word_tokenize(document))

remove duplicates so only UNIQUE words remain
all_words_lower_case = [word.lower() for word in all_words]
unique_words = sorted(list(set(all_words_lower_case)))

Process out stop words to create VOCABULARY
stop_words = set(stopwords.words('english'))
unique_words = filter(lambda word: word not in string.punctuation, unique_words)
vocabulary = [element for element in unique_words if element.casefold() not in stop_words]

total_documents = len(documents)

vectors = []

for document in documents:
doc_vec = np.zeros((len(vocabulary),)) # create vector of all 0's
index = 0
for word in vocabulary:
# calculate TF
occurences = len([token for token in document.split() if token.lower() == word])
words_in_doc = len(document.split())
tf = float(occurences/words_in_doc)

```
  # calculate IDF
    occurences = 0
    for inner_document in documents:
        if word in inner_document.lower():
          occurences += 1

    idf = np.log10(total_documents/occurences)

    # store IDF into the vector
    doc_vec[index] = tf * idf
    index += 1
  vectors.append(doc_vec)

print("{0} \n{1}\n".format(vocabulary,np.array(vectors)))
```

Scikit-Learn: TfidfVectorizer

Similar to the `CountVectorize` functionality for a bag of words model, Scikit-learn has a `TfidfVectorizer` class (see full documentation here).

To create a bag of words model using TF-IDF scoring, simply initialize the `TfidfVectorizer` and feed in the documents:

```
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
```

To see the result, you can use:

```
print(vectorizer.get_feature_names_out()) # prints vocabulary
print(X.toarray()) # prints TF-IDF scores
```

Expected Output:

```
['ball', 'cat', 'dog', 'plays', 'the', 'with']
[[0.33379109 0.         0.46913173 0.33379109 0.66758217
0.33379109]
 [0.33379109 0.46913173 0.         0.33379109 0.66758217
0.33379109]]
```

important

Those numbers look different!

We have been using the same documents across the past few pages – but these TF-IDF values are different then before!

`TfdifVectorizer` has a number of mechanisms to refine the TF-IDF calculation, including `smooth_idf` which defaults to `True`. `smooth_idf` pretends there is an extra document containing every word in the vocabulary to prevent division by zero errors.

You can read more about how Scikit does the TF-IDF calculation

Notice that a number of design decisions have been made for us by `TfidfVectorizer`. For example:
1. All words are forced to lower case (`lowercase=True`)

1. The vocabulary is full of single words (`analyzer='word'`)
1. Tokens are made up of 2 or more alphanumeric characters (punctuation is completely ignored)
1. Stop words are not filtered out (`stop_words=None`)

These assumptions about design decisions can be overridden.

challenge

Try these variations:

- Try removing stop words:
  `python vectorizer = TfidfVectorizer(stop_words='english')`
- Try turning off the `smooth_idf` functionality:
  `python vectorizer = TfidfVectorizer(smooth_idf=False)`

N-grams example

Let's create a TF-IDF scored model with 2-grams:

```
vectorizer = TfidfVectorizer(analyzer='word', ngram_range=(2,
        2))
X = vectorizer.fit_transform(documents)
```

We can then print out the vocabulary to see the 2-grams:

```
print(vectorizer.get_feature_names())
```

We can also print out the vectors to see the usage:

```
print(X.toarray())
```

Formative Assessment 1

Formative Assessment 2